



Table of Contents

1. Creating and Managing Custom Activities	5
1.1. Introduction to Custom Activity.....	5
1.2. Creating New Custom Activity Configuration	5
1.2.1. Creating Subflow for Custom Activity	7
1.2.2. Custom Activity Connector Types	8
1.3. Viewing	8
1.4. Duplicat	9
1.5. Exportin	10
1.6. Importin	11
1.7. Deleting	14
2. Custom Act	17
2.1. Adding a	17
2.2. Custom	17
2.2.1. C	18
2.2.2. C	20
2.3. Connect	20
3. Appendix	23
3.1. JSON Co	23
3.1.1. J	23
3.1.2. J	27
3.2. JavaScri	30
3.2.1. C	30
3.2.2. Javascript Code Example	31
3.3. JAVA Code Details	49
3.3.1. Installation of Platform JAR Dependencies.....	49
3.3.2. Supporting Files for Custom Activity	50
3.3.3. Node JAR Details	50
3.3.4. Uploading the Node JAR for Custom Activity	52
4. References	53
4.1. Supporting Documents	53

1. Creating and Managing Custom Activities

1.1. Introduction to Custom Activity

Custom activities are activities that can be designed as per your requirements. The activities created can be utilized in the process flows for an organization.

There are two types of custom activities:

- a) custom activity of type custom activity,
- b) custom activity of type subflow.

A custom activity of type custom activity is designed by uploading JSON files, Javascript, and JAR files that contain the configuration details required for that custom activity.

A custom activity of type subflow allows you to design a process flow as an activity.

1.2. Creating New Custom Activity Configuration

Follow the below steps for creating a new Custom Activity configuration.

1. Click the Burger menu and navigate to **Manage > Configuration Management**.
1. Click **Custom Activity** in the configuration entity panel.

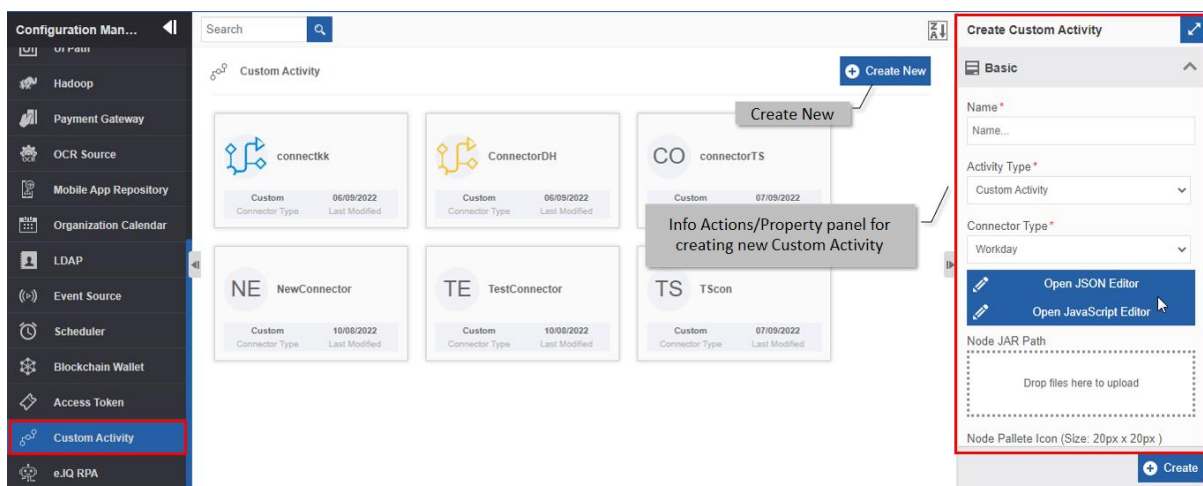


Figure 1.1: Creating new Custom Activity

2. Click **+Create New**.
3. Enter the configuration details in the **Create Custom Activity** panel as explained below.

Field	Description
Name*	Enter the name of the custom activity. Character limit: 50. Data type: Alphanumeric and underscore.
Activity Type*	Select the activity type.

Field	Description
	Activity Type * Custom Activity Custom Activity SubFlow Activity Custom as per your configuration details. Sub Flow can be used as an activity. Refer to
Connector Type*	Select the Connector Custom Workd HighQ UIPath Custom My Info Swift Blue P Chat G Refer to more details on connectors.
Open JSON Editor	Click to The info in the JS The nod Activity Refer to for more information.
Open JavaScript Editor	Click to The det the java Refer to tails for more information.
Node JAR Path	Upload The pro custom Refer to ails for more information.
Node Pallete Icon*	Upload appears Only SV graphics The ma loaded is 20px X 20 px
Node Icon*	Upload when th area. activity in the designer area to the process flow designer

Field	Description
	Only SVG file format is supported. SVG is used to define vector-based graphics for the Web. The maximum size of the image that can be uploaded is 40px X 40px
Node Accordion Icon	Upload node accordion icon. This icon appears on the Info Actions panel for the activity configuration category accordion. Only SVG format is supported. SVG is used to define vector-based graphics for the Web. The maximum size of the image that can be uploaded is 20px X 20 px.
Description	Write a brief description of the custom activity. Character limit: 1000 characters Data type: Alphanumeric and symbols.

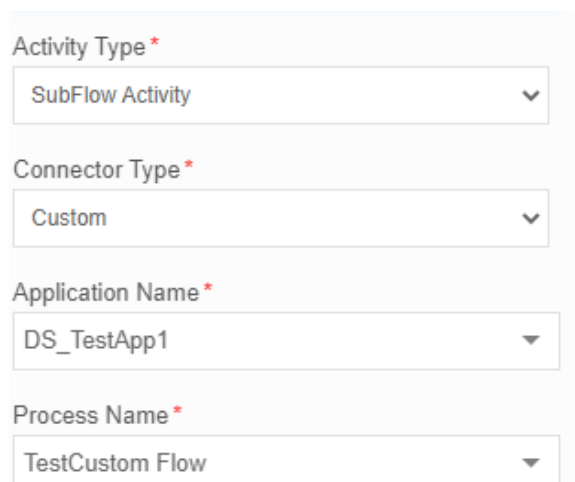
 * Mandatory (required) fields.

- Click **Create** on the bottom right of the page and the custom activity gets created with the details entered.

1.2.1. Creating Subflow for Custom Activity

To create a process flow as a custom activity, follow the below steps.

- Navigate to **App Studio > Process Flow** module.
- Create and configure the process flow details and entities as per your needs.
- Deploy the process flow.
- Navigate to **Manage > Configuration Management > Custom Activity**.
- Click *Create New* and create a Custom Activity by selecting **Activity Type** as **Sub Flow Activity**. Application Name drop-down is displayed. The application name is displayed only if one of the process flow within that application is deployed.
- Select the application name. The Process flow list is displayed. Only the deployed process flows are listed in the drop-down.
- Select the process flow that you want to define as the custom activity.



Activity Type *

SubFlow Activity

Connector Type *

Custom

Application Name *

DS_TestApp1

Process Name *

TestCustom Flow

- For other configurations, go to step 3 details and configure accordingly, and finally click **Create** for creating the subflow type custom activity.

1.2.2. Custom Activity Connector Types

Connectors are used to achieve specific requirements in your activity. Some connectors are part of the Platform, however, you can include external connectors for utilizing the features of the same.

The connector types supported in the custom activities are explained below.

Blue Prism: Selecting Blue Prism allows you to create a custom activity that interacts with the Blue Prism application via APIs. When you create a Blue Prism custom activity, it will be listed in the Blue Prism category of the Activities.

Chat GPT: Selecting Chat GPT allows you to create a custom activity that interacts with the Chat GPT application via APIs. When you create a Chat GPT custom activity, it will be listed in the Chat GPT category of the Activities.

Custom: Select the option “Custom” if you are defining a custom activity or if you are creating a subflow custom activity within the Platform.

HighQ: Selecting HighQ allows you to create a custom activity that interacts with the HighQ application via APIs. When you create a HighQ custom activity, it will be listed in the HighQ category of the Activities.

My Info: Selecting My Info allows you to create a custom activity that interacts with the My Info application via APIs. When you create a My Info custom activity, it will be listed in the My Info category of the Activities.

Swift: Selecting Swift allows you to create a custom activity that interacts with the Swift application via APIs. When you create a Swift custom activity, it will be listed in the Swift category of the Activities.

UIPath: Selecting UIPath allows you to create a custom activity that interacts with the UIPath application via APIs. When you create a UIPath custom activity, it will be listed in the UIPath category of the Activities.

Workday: Selecting Workday allows you to create a custom activity that interacts with the Workday application via APIs. When you create a Workday custom activity, it will be listed in the Workday category of the Activities.

1.3. Viewing and Editing Custom Activity

1. Click the Burger menu and navigate to **Management > Configuration Management > Custom Activity**.
2. Click the Custom Activity card to view the details of the selected Custom Activity. The details of the Custom Activity appear in the Info Actions panel (**Edit Custom Activity**).

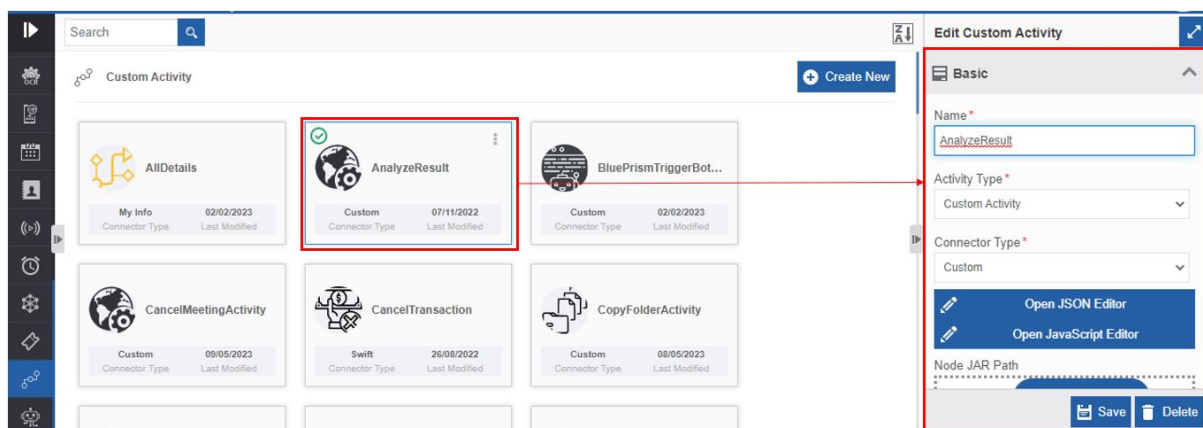


Figure 1.2: Editing Custom Activity details

3. Edit the Custom Activity details as needed.
4. Click **Save**.

1.4. Duplicating Custom Activity

Follow the below steps for duplicating an existing custom activity.

1. Click the Burger menu and navigate to **Manage > Configuration Management**.
2. Click **Custom Activity**. The list of all custom activities is displayed.
3. Hover over any custom activity card. Three dots appear on the upper right side of the card.
4. Click the three dots. More Actions appear.

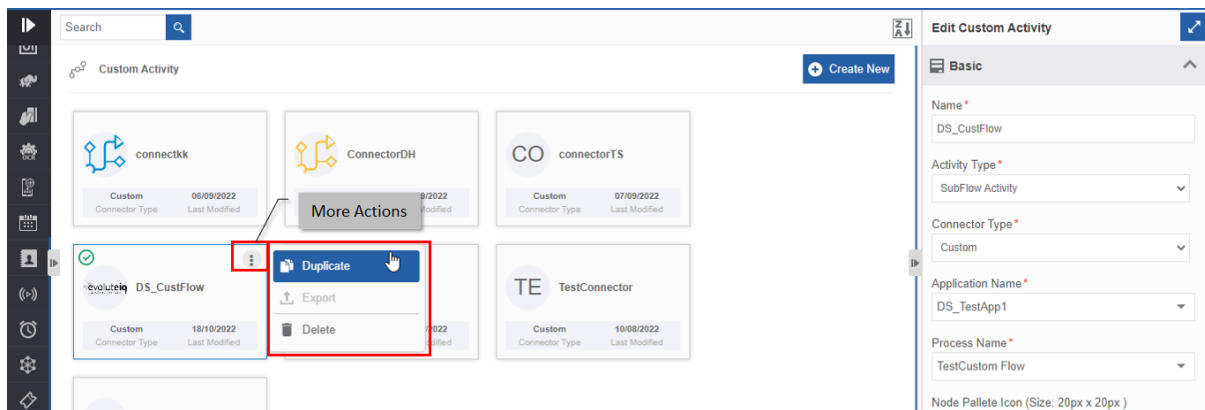


Figure 1.3: Duplicating the custom activity

5. Click **Duplicate**. A confirmation pop-up appears.

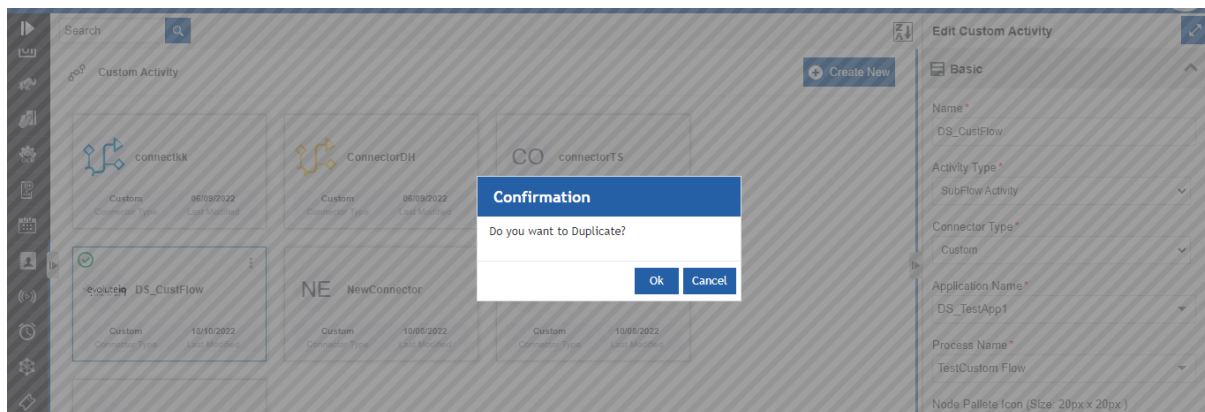


Figure 1.4: Duplicate confirmation

6. Click **Ok** for duplicating the custom activity (or you can click **Cancel** to cancel the duplicate action). A **Success** message appears on the successful duplication of the custom activity.

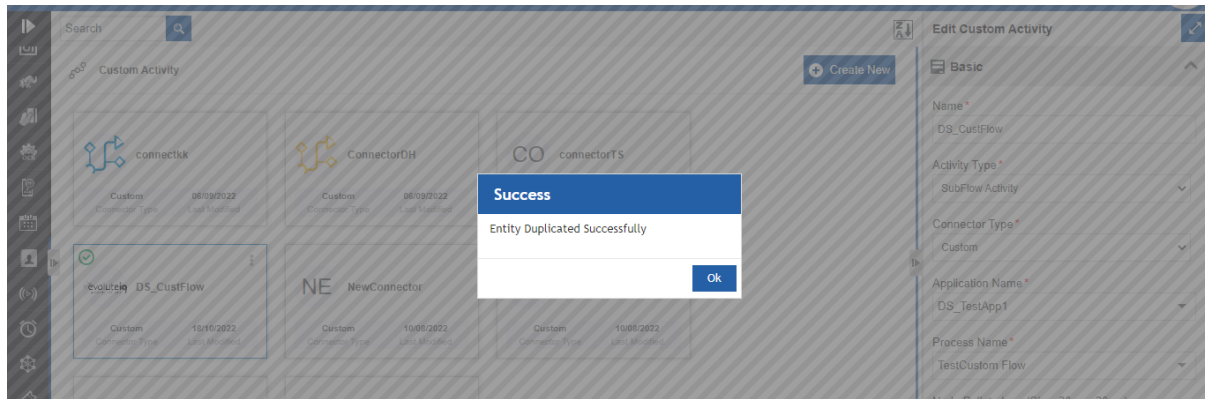


Figure 1.5: Duplicate success message

7. Click **Ok**. A duplicate copy of the custom activity appears on the custom activity page with the same custom activity name suffixed with “**_copied**”.

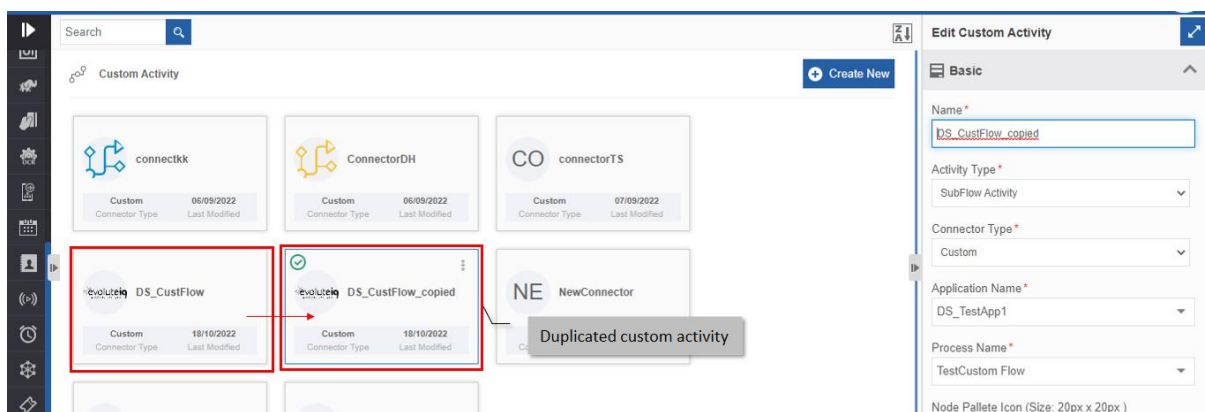


Figure 1.6: The duplicated custom activity

1.5. Exporting a Custom Activity

You can export a custom activity as a package to your local system.

1. Click the Burger menu and navigate to **Management > Configuration Management > Custom Activity**.
2. Hover over a custom activity name card and click the three dots (More Actions).

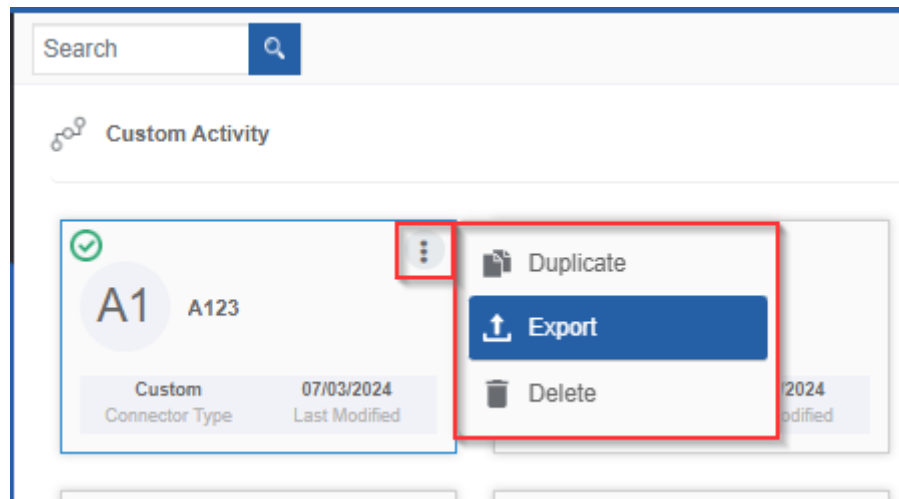


Figure 1.7: Exporting Custom Activity

3. Click **Export**. The selected custom activity gets exported to your local system in the .pkg format.

1.6. Importing Web App

You can import a custom activity package into the Custom Activity section for utilization.

1. Click the Burger menu and navigate to **Management > Configuration Management > Custom Activity**. The list of all custom activities in the platform appears.

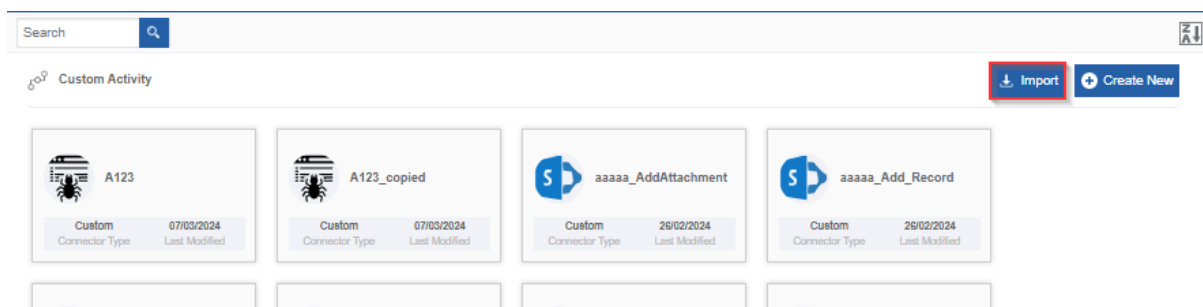


Figure 1.8: Import custom activity

2. On the upper right click **Import**.

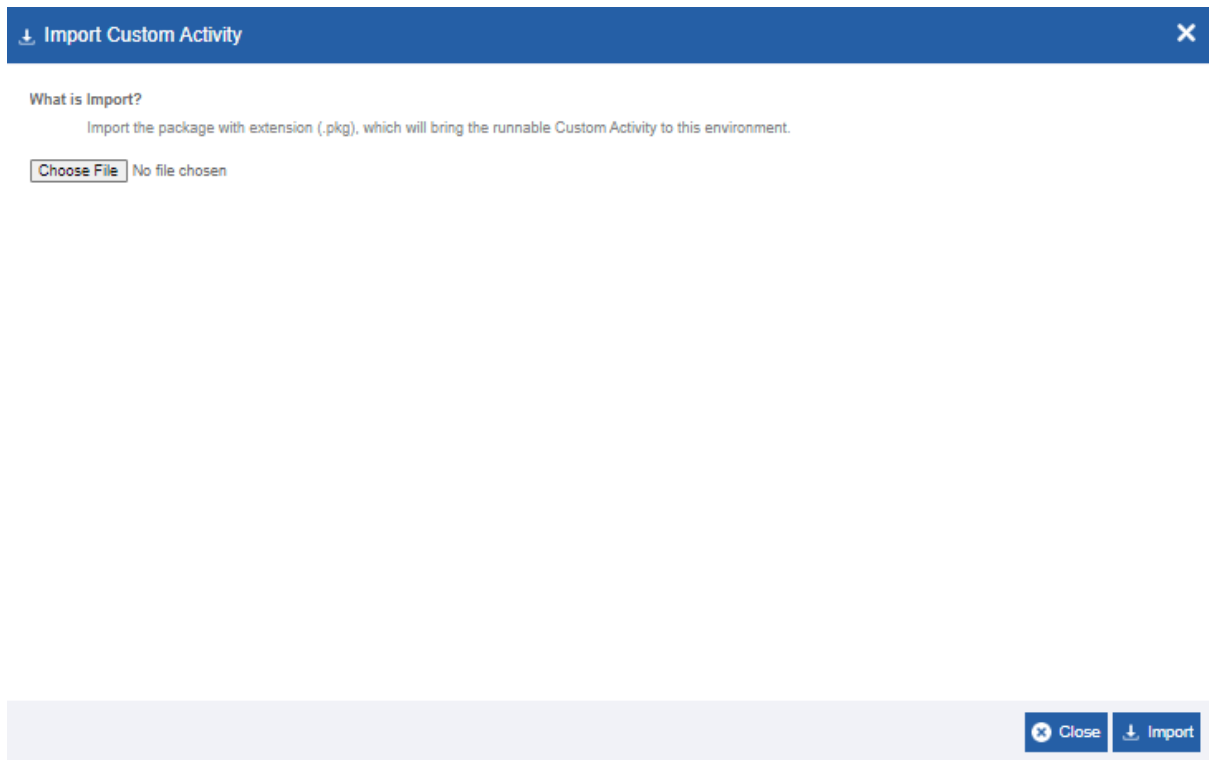


Figure 1.9: Importing Custom Activity - file selection

3. Click **Choose File** and select the file (in .pkg format) that you want to import. If the import package name and matches with any of the existing custom activity, import failed message appears with an option to rename the package name or to overwrite the existing custom activity in the platform.

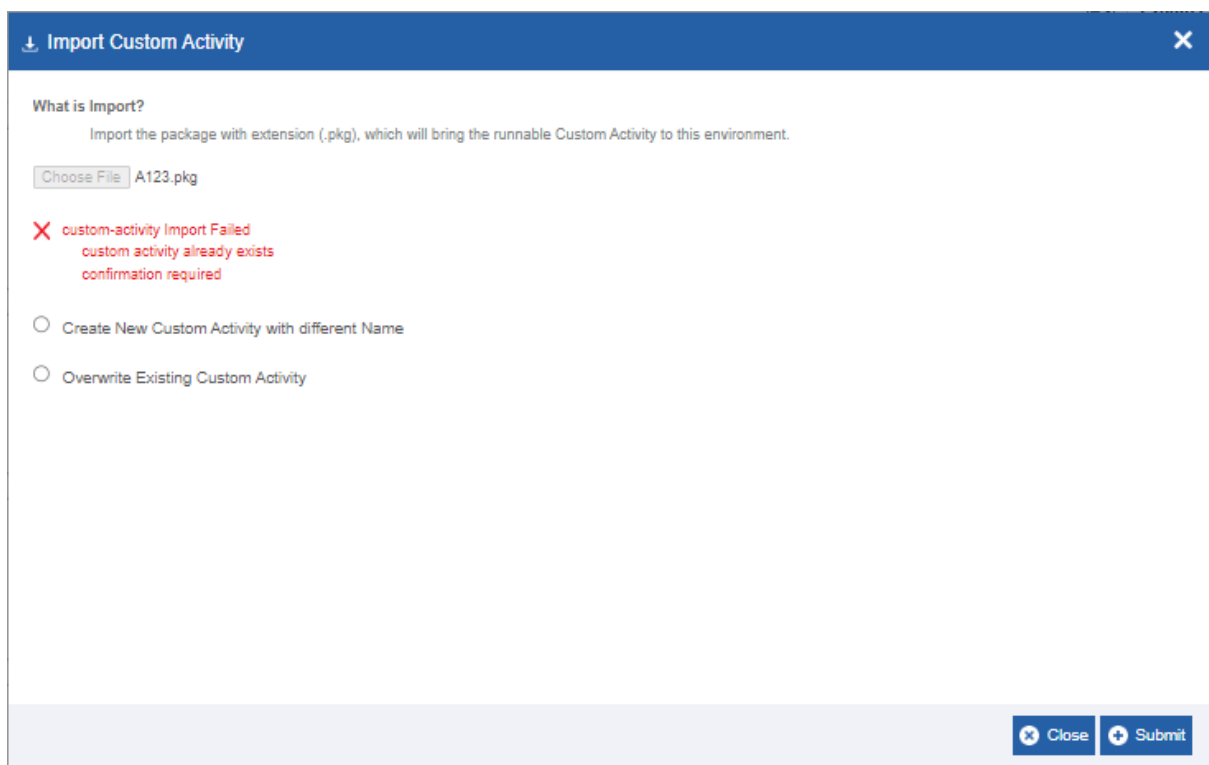


Figure 1.10: Import custom activity import failed message with options

- **Create new Custom Activity with different name:** Click this option if you want to import the same package with a different name.

↓ Import Custom Activity

What is Import?
Import the package with extension (.pkg), which will bring the runnable Custom Activity to this environment.

Choose File | A123.pkg

✗ custom-activity Import Failed
custom activity already exists
confirmation required

☒ Create New Custom Activity with different Name
☐ Overwrite Existing Custom Activity

Enter Name
Import the package (.pkg) with new name provided below and then click on Submit.

✕ Close

➕ Submit

Enter the new name in the **Enter Name** field.

Overwrite Existing Custom Activity: Click this option to overwrite the existing custom activity with the import file with the same name as in the package.

4. Click **Import**. A Success message appears on successful import of the package.

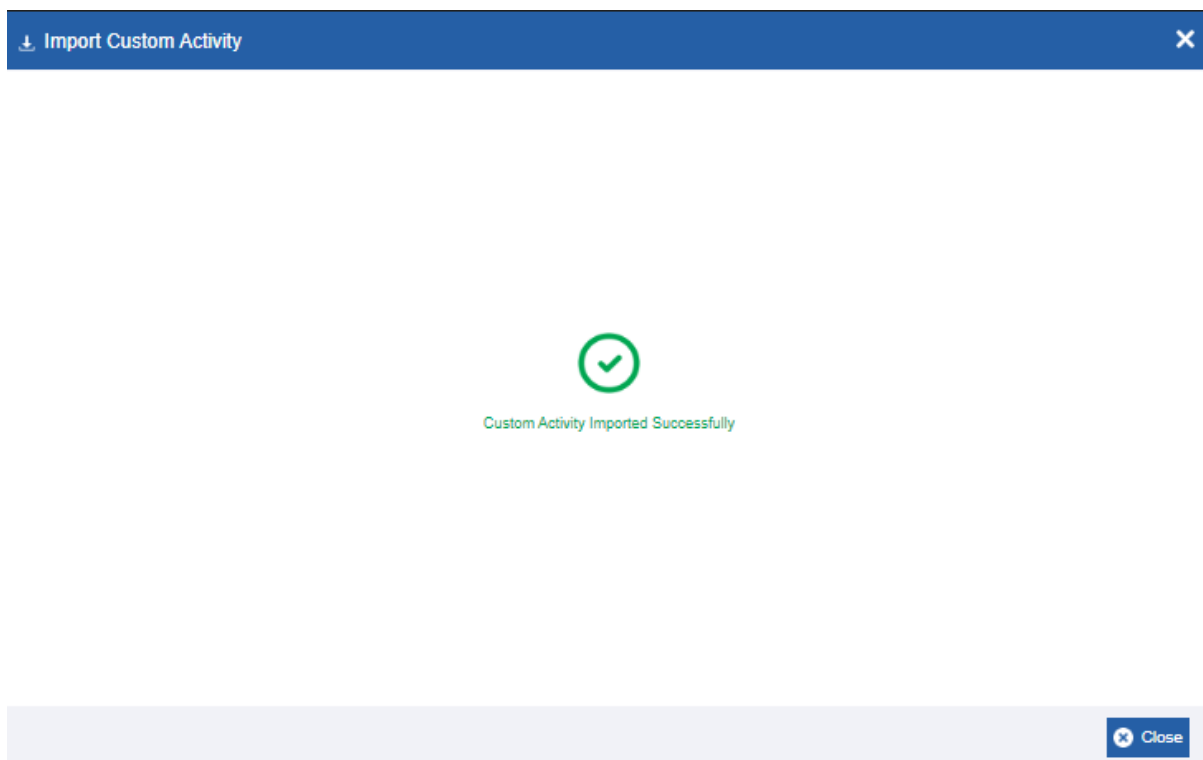


Figure 1.11: Import success message

5. Click **Close**. The package is imported to the platform.

1.7. Deleting Custom Activity

1. Click the Burger menu and navigate to **Manage > Configuration Management**.
2. Click **Custom Activity**. The list of all custom activities is displayed.
3. Click the custom activity name card that is to be deleted. The lower-right of the page displays the **Delete** button.

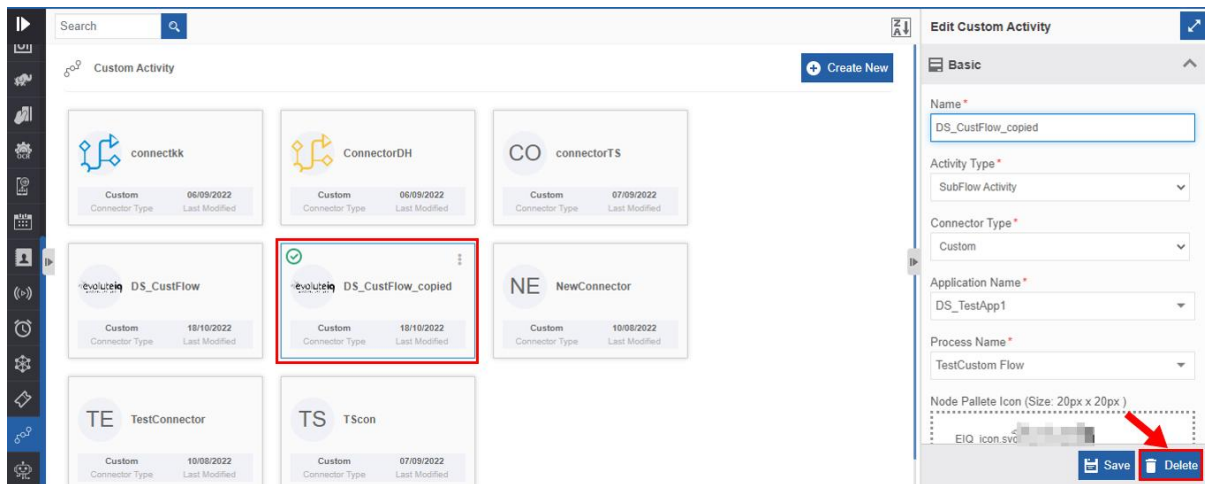


Figure 1.12: Deleting custom activity

4. Click **Delete**. A **Confirmation** pop-up for delete appears.

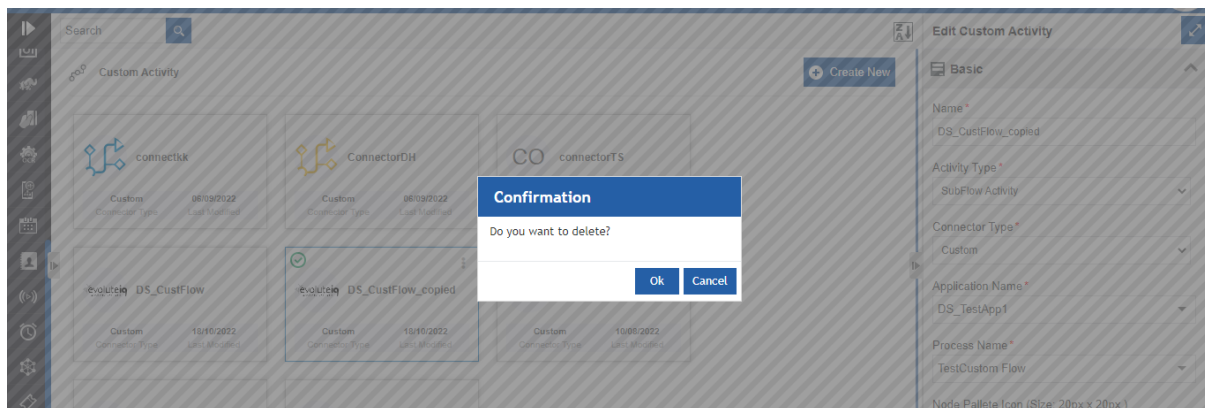


Figure 1.13: Custom Activity Delete confirmation

5. Click **Ok** for deleting the custom activity.

Or

Click **Cancel** to cancel the action.

Alternatively, you can follow the below steps to delete the custom activity:

- i. Click the Burger menu and navigate to **Manage > Configuration Management > Custom Activity**
- ii. Hover over the custom activity card. Three dots appear on the upper right side of the card.
- iii. Click the three dots. More Actions appear.

iv. Click **Delete** and follow step 5 in the above procedure.

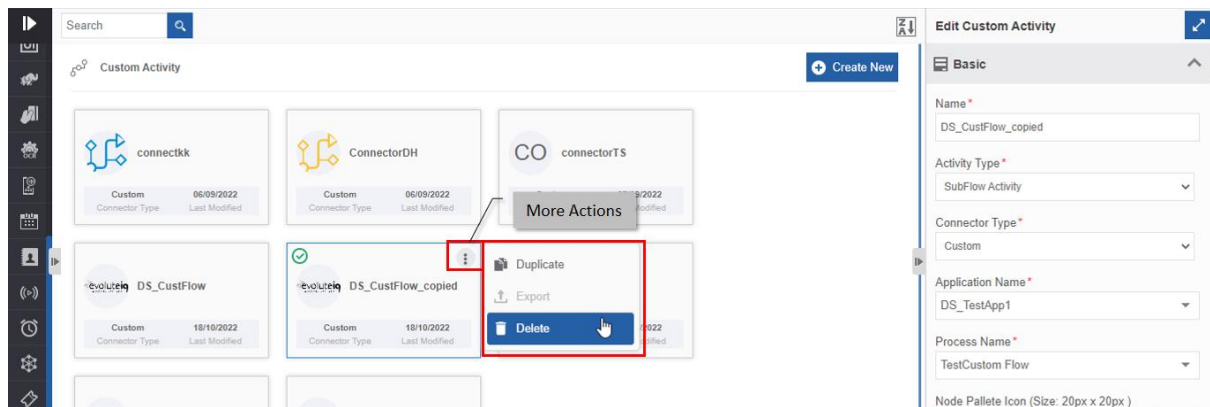


Figure 1.14: Delete action in More Actions

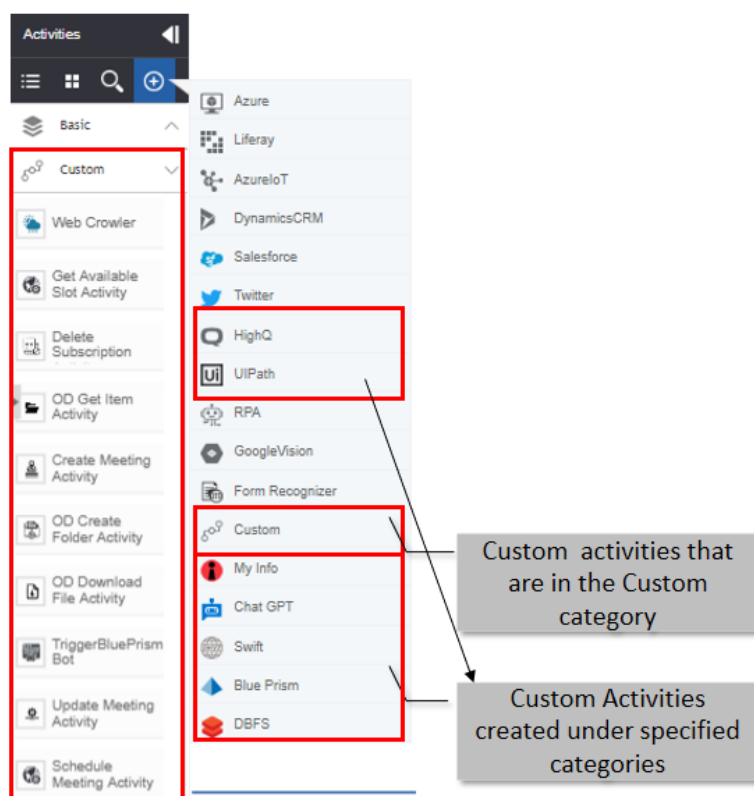
2. Custom Activities in Process Flows

2.1. Adding and Viewing Custom Activities

The custom activities created in the Manage > Configuration Management > Custom Activities can be accessed through the process flow Activities panel.

1. Navigate through **App Studio > Process Flows**.
2. In the Process Design > **Activities** panel click “+” for viewing the advanced connectors.
3. Click **Custom**. All the custom-defined activities get added to the Activities panel under the **Custom** accordion.

The custom activities that are of type “Custom” are displayed below the Custom category. Other custom activities are listed below the selected custom category type. Custom activities that are in different categories will be listed under that category along with the default activities (if any) in that category.



2.2. Custom Activity in Process Flow

To use the custom activity, you need to add the custom activity category to the activities list and then drag the activity to the process designer.

Based on the type of custom activity added (Sub Flow Activity Type or Custom Activity Type) to the process flow, the configuration details are different.

The sections included in Info Actions for the custom activity of type Subflow are Basic, Trigger Subflow, and Error Handling. Refer to [Configuring Subflow Activity](#).

The sections included in Info Actions for the Custom activity of type custom activity are Basic and the configurations are coded as in the JSON file. Refer to [Configuring Custom Activity](#).

2.2.1. Configuring Subflow Activity

1. Add the Custom Activity category to the Activity list. Refer to [Adding and Viewing Custom Activities](#).
2. Design the process flow by dragging the subflow custom activity as needed.

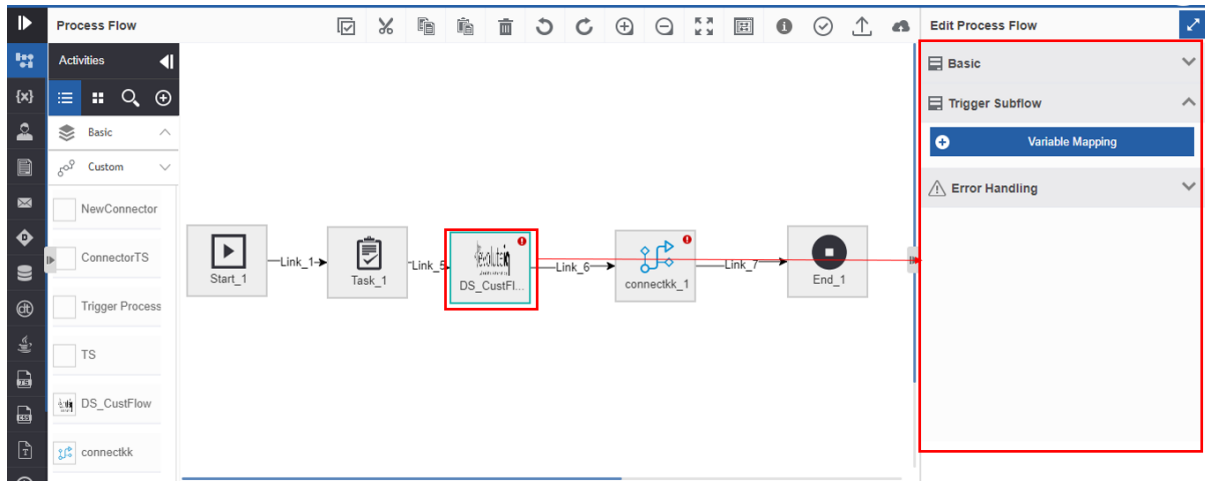


Figure 2.1: Custom subflow activity in process flow and the configurations

3. Click the activity. Configuration details are displayed on the Info Actions panel.
4. Click **Basic** accordion.
5. Provide **Name** and **Description** for the activity.

Basic

Name *

DS_CustFlow_1

Description

Description about the activity

6. Click **Trigger Subflow** accordion.

Basic

Trigger Subflow

+ Variable Mapping

Error Handling

- i. Click **+ Variable Mapping**. Variable mapping pop-up appears.

The Variable Mapping dialog box is divided into two main sections: 'Parent Process To Child Process' and 'Child Process To Parent Process'. Each section contains a table with columns for 'Input Variable', 'Output Variable', and 'Action'.

Parent Process To Child Process		
Input Variable	Output Variable	Action
Var1	DS_Form1.Name	[Delete Icon]
+ Variable		

Child Process To Parent Process		
Input Variable	Output Variable	Action
Task_1.assignedToSwimlanes	Var1	[Delete Icon]
+ Variable		

At the bottom right, there are buttons for 'Close' and 'Save'.

Figure 2.2: Variable Mapping

- ii. In the Parent Process To Child Process, click **Variable** and select **Input Variable** (parent process variable) and **Output Variable** (child process variable). This is done to pass the data from the parent process to the child process, that is, custom subflow activity.
 - iii. In the Child Process To Parent Process, click **Variable** and select **Input Variable** (child process) and **Output Variable** (parent process). This is done to pass the data from the child process, that is custom subflow activity to the parent process.
 - iv. Click **+ Variable**, to add more variables in each section.
 - v. Click the delete icon to remove an input or output variable mapping.
 - vi. Click **Save**.
7. Click **Error Handling** accordion.

The Error Handling accordion is expanded, showing the following options:

- ☐ **Continue If Error**
- Error Variable**
- Error Variable (dropdown menu)

Figure 2.3: Error Handling

- i. Check the **Continue If Error** check box if you want to continue the workflow even if the error occurs. Else uncheck the checkbox to break or stop the workflow execution when an error occurs.
 - ii. Select a variable from the **Error Variable** drop-down. The variable should be of data type, alphanumeric. Error variable stores the first line of the error.
8. Click **Create/Save** to save the changes.

2.2.2. Configuring Custom Activity

Custom activity configurations depend on the JSON file, JavaScript file, and the JAR file that you uploaded while creating the custom activity in the **Manage > Configuration Management > Custom Activity**.

1. Add the Custom Activity category to the Activity list. Refer to [Adding and Viewing Custom Activities](#).
2. Design the process flow by dragging the custom activity as needed. The custom activities appear below the Custom category in the Activities pane. The name of the custom activity is the node name that you have coded in the JSON file.

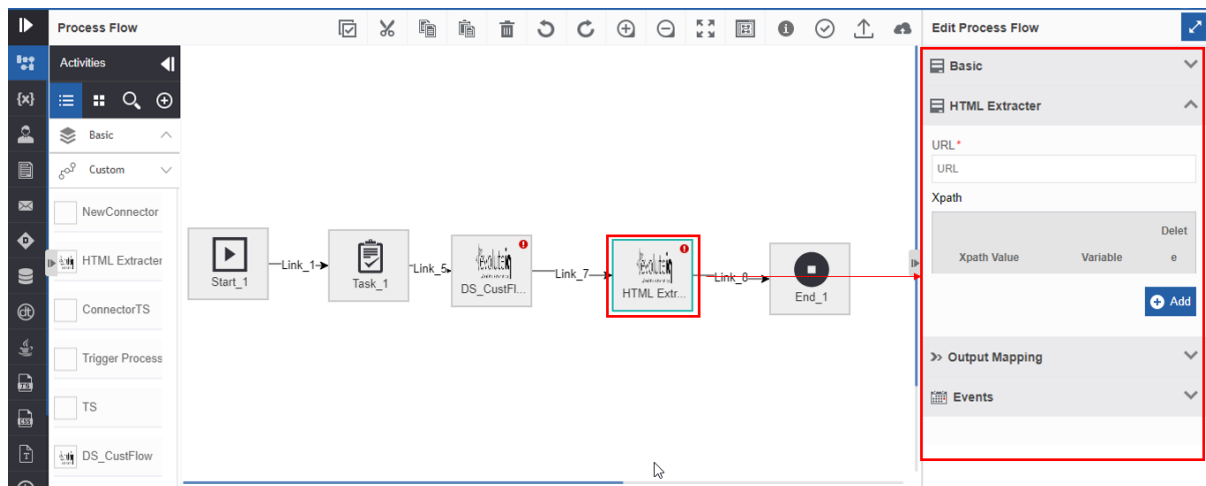


Figure 2.4: Custom activity in process flow and the configuration detail in the Info Actions

3. Click the activity. Configuration details are displayed on the Info Actions panel.
4. Click **Basic** accordion.
5. Provide **Name** and **Description** for the activity.

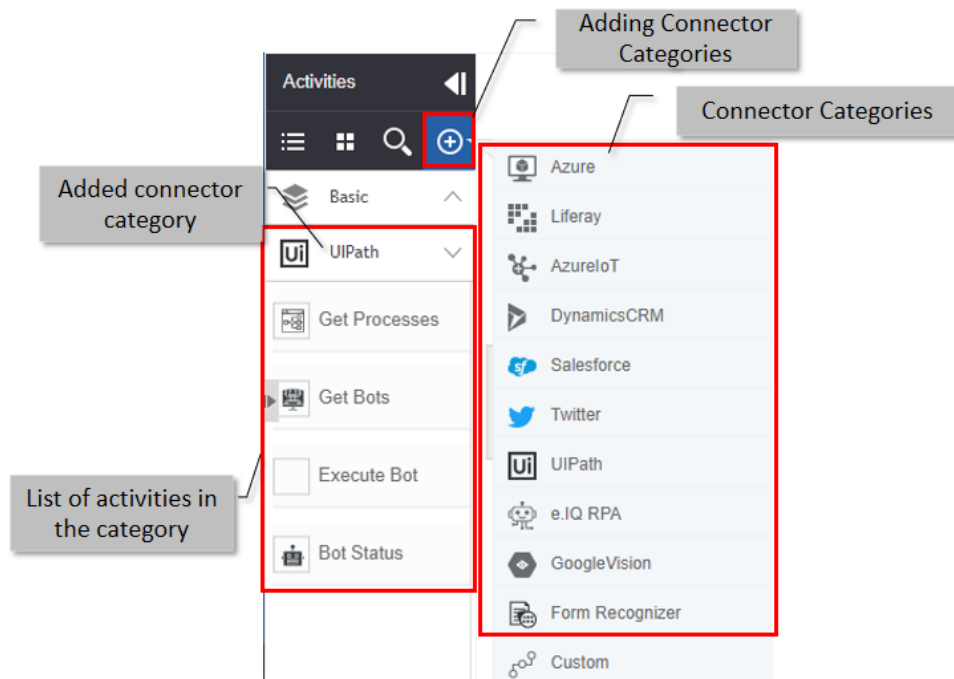
 This screenshot shows the 'Basic' configuration section of the custom activity. It features two input fields: 'Name' with the value 'DS_CustFlow_1' and 'Description' with the placeholder text 'Description about the activity'. The section is titled 'Basic' with an expand/collapse arrow.

6. Click **{custom_configuration}** accordion (that appears on the Info Actions) and configure the details as needed. The custom configuration accordion and details appear based on JSON file details that you provided in the Manage module.

2.3. Connector-Based Custom Activities

The connector-based custom activities are displayed on the specific Connector category in the Activities panel. The connector-based activities (default) and the custom activities can be accessed only by adding those activities to the Activities panel.

- In the Process Flow Activities Panel > Click “+” > click the connector name. All the activities supported for that connector including the custom activities are displayed below the connector category.



3. Appendix

3.1. JSON Code Details

3.1.1. JSON Code Content

Prepare Node JSON as mentioned below.

 Refer to CustomActivity.json file for sample json format.

Update the following properties in the JSON Content in the JSON Editor and save.

•  handler class name with full package as mentioned below.
Exa  cterHandler"

•  property class name with full package as mentioned below.
Exa  cterMapping"

•  Property class name with full package as mentioned below.
Exa  cterValidator"

By  a node configuration: Input Configuration, Output/Error
Cor  ecated)

Input

Let  e name of the Section.
"name": Name of the Section, it should be same as a node name.
"tab": Name of the Section without spaces, it will be the accordion name.
"widget": We will define a widget under the widget list.

Each widget will have the following configuration.

"type": "property-form-custom"

"triggerFunction": "init"

"data": data will have single/multiple objects based on our requirement.

Basically, these are input fields for a Node.

We will define one custom input element under the input section.

```

{
  // Since it is a custom control, we will create an element as div.
  "properties": {
    // Some description of the control
    "id": "id of the control", // id of the control, DOM, value
    "name": "name of the control", // name of the control, DOM.
    "object": "object of the control", // object of the control, DOM.
    // Name of the control, which will be used in
    // the JavaScript file to handle DOM changes.
    "control": "control",
    // this is the control name, which will be used in
    "priority": "priority"
  }
}

```

Custom Control widget is defined in JavaScript file to handle the DOM changes, events and data management.

Now we will create a Selectize component for listing the configured data sources.

```

{
  // selected value
  // properties of the element
  // id of the control, DOM, value
  // management
  // same
  //label
  //there is a method to handle the
  //different
  //type of the control, which is defined in the platform
  //autocomplete
  //title
  //placeholder
  //element
  //event type
  "action": "change",
  // default call back method
}

```



```

        "callback": "save"
    }
},

```

Now we will create a normal textbox with droppable variable.

```

{
  //input text element
  "element": {
    "type": "text",
    //label of the element
    "label": "Variable",
    //description of the element
    "description": "Variable",
    //id of the element
    "id": "variable",
    //class of the element
    "class": "variable",
    //validation classes
    "validation": {
      "required": true,
      "min": 1,
      "max": 100,
      "pattern": null,
      "custom": {
        "imagePath": null,
        "name": "variable",
        "placeholder": "Variable",
        "type": "text",
        "title": "Variable",
        "default": "",
        "element": "variable",
        "variable": "variable",
        "will be allowed"
      }
    }
  },
  "event": {
    //event type
    "type": "click",
    // default call back method
    "callback": "save"
  }
}

```

Output/Error Configuration

Use below JSON to create the output and error elements to handle the extracted data or failure cases. This will be commonly used for any custom activity.

```

{
  "name": "Output Mapping",
  "tab": "outputMapping",
  "widget": [
    {
      "type": "property-form-custom",

```

```

    "data": [
      {
        "elementType": "object",
        "properties": {
          "label": "Object",
          "description": "Object",
          "id": "object",
          "name": "Object",
          "objectType": "object",
          "class": "object",
          "fieldId": "object",
          "type": "object",
          "placeholder": "Object",
          "title": "Object",
          "priority": 1
        },
        "event": {
          "action": "change",
          "callback": "save"
        }
      },
      {
        "elementType": "array",
        "properties": {
          "label": "Array",
          "description": "Array",
          "id": "array",
          "name": "Array",
          "objectType": "array",
          "class": "array",
          "fieldId": "array",
          "type": "array",
          "placeholder": "Array",
          "title": "Array",
          "priority": 1
        },
        "event": {
          "action": "change",
          "callback": "save"
        }
      }
    ]
  },

```

Event Configuration (Deprecated-check before using the code)

We can use the below JSON to create event configuration.

```

{
  "name": "Events",
  "tab": "property-events",

```

```

    "widget": [
      {
        "type": "property-form",
        "data": [
          {
            "elementT
            "properti
            {
              "id": "
              "name":
              "placeh
              "type":
              "compon
              "compon
              "title"
              "priori
            },
            "event":
            {
              "action": "change",
              "callBack": "save"
            }
          }
        ]
      }
    ]
  }

```

3.1.2. JSON Code Example

```

{
  "nn": "HTML Extracter",
  "nd": "HTML Extracter",
  "loc": "",
  "tp": "svg",
  "it": "true",
  "ot": "true",
  "mit": "true",
  "mot": "true",
  "txt": "HTML Extracter",
  "sts": "",
  "fp": "images/process-designer-node-
icons/nodes/svg/UnitConverter.svg",
  "canvas":
icons/nodes/
  "type": "v
  "grp": "Ba
  "group": "
  "ht": "fal
  "nodegroup
  "otc": "1"
  "CKey": "r
  "notValid"
  "grp_icon
  "grp_icon
icons/groups/svg/UnitConverter.png",

```

```
"node_handler": "com.demod",
"node_enter_handler": "",
"node_leave_handler": "",
"propertyClass": "com.demong",
"node_validator_class":
"com.demol.custom.HtmlExtra
"default_section": "1",
"widget-namespace": "NBT.
"section": [
  {
    "name": "HTML Extract
    "tab": "HTMLExtractor
    "widget": [
      {
        "type": "property
        "triggerFunction"
        "data": [
          {
            "elementType"
            "properties":
              "label": "U
              "id": "url"
              "class": "i
              "name": "ur
              "placeholder
              "maxlength"
              "type": "te
              "title": "U
              "priority":
            }, "event":
              {
                "ad
                "ca
              }
          },
          {
            "elementType"
            "properties":
              "descriptio
              "id": "xpat
              "name": "xp
              "objectType
              "controlWid
              "priority":
          }
        ]
      }
    ]
  },
  {
    "name": "Output Mapping",
    "tab": "outputMapping",
    "widget": [
      {
```

```
"type": "property-form",
"data": [
  {
    "elementType": "property-form",
    "properties": {
      "label": "Property",
      "description": "Property description",
      "id": "property",
      "name": "property",
      "objectType": "property",
      "class": "property-form",
      "fieldId": "property",
      "type": "property-form",
      "placeholder": "Property placeholder",
      "title": "Property title",
      "priority": 1
    },
    "event": {
      "action": "property-action",
      "callback": "property-callback"
    }
  },
  {
    "elementType": "property-form",
    "properties": {
      "label": "Property",
      "description": "Property description",
      "id": "property",
      "name": "property",
      "objectType": "property",
      "class": "property-form",
      "fieldId": "property",
      "type": "property-form",
      "placeholder": "Property placeholder",
      "title": "Property title",
      "priority": 1
    },
    "event": {
      "action": "property-action",
      "callback": "property-callback"
    }
  }
]
},
{
  "name": "Events",
  "tab": "property-events",
  "widget": [
    {
      "type": "property-form",
      "data": [
```

```

    {
      "elementType": "text",
      "properties": {
        "id": "event",
        "name": "Event",
        "placeholder": "Enter event name",
        "type": "text",
        "component": "Form",
        "component": "Form",
        "title": "Event",
        "priority": "High"
      },
      "event": {
        "action": "change",
        "callBack": "save"
      }
    }
  ]
}
]
}
}

```

3.2. JavaScript Code Details

3.2.1. Control Widget JavaScript File

Define the Control widget as mentioned below, this is the default definition of the Control widget.

 Refer to CustomActivity.js file for sample javascript format.

```

$.widget("nbt.CustomKeyValuePairWidget", $.nbt.ControlWidget,
{
  options: {
    //Define the options required to be reused in the widget
    processVar: "processVar",
  },
  _create: function() {
    //Initialize a new widget
  },
  render: function() {
    //define html element to create an
    element in the
  },
  _init: function() {
    //This method is initialised and
    available.
  }
});

```

3.2.2. Javascript Code Example

```
$.widget("nbt.CustomKeyValuePairWidget", $.nbt.ControlWidget,
{
    options: {
        processVariables: true,
    },
    _create: function() {
        this.objectType =
        this.options.value.proper
        this._readOnly =
        (this.options.value.prope
        : false;
        this._uniqueId =
        this.constants =
        _labels: {
            "table":
            "variable
            "key"
            "valu
            "head
            "addB

        },
        "customAt
            "key"
            "valu
            "head
            "addB
        },
        "headers"
            "key"
            "valu
            "head
            "addB
        },
        "xpathVar
            "key"
            "valu
            "head
            "addB
        },
        "paramete
            "key"
            "valu
            "head
            "addB
        },
        "formFields": {
            "key": "Key",
            "value": "Value",
```

```

        "headers": "Form Fields",
        "addB
    },
    "formFiel
        "key"
        "valu
        "head
        "addB
    },
    "customQu
        "key"
        "valu
        "head
        "addB
    },
    "columns"
        "key"
        "valu
        "head
        "addB
    },
    "inputPar
        "key"
        "valu
        "head
        "addB
    },
    "deletela
    "name": "
    "dataType
    "primaryK
    },
    _tooltips:{
        "primaryKey"
    }
};
if(this.objectType
    this._getDat
}
if(this.objectType
    this._getTak
}
this.render();

},

/**
 * Loads all the temp
 */

render: function() {
    this.tpl_wrapper = service-
headers-rest form-row" style="display: inline-block;padding:
0px;width: 100% !important;"></div>');
    this.tpl_div = _.template('<div></div>');

```



```

        this.tp
style="width: 400px; height: 20px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
        this.tp
style="width: 400px; height: 20px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
        this.tp
style="padding: 5px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
class="<%=cssClass%>"
        this.tp
id = "mapper-component-table-component"
        this.tp
class="variable"
name%>" style = "width: 400px; height: 20px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
        this.tp
style="margin-bottom: 10px; border: 1px solid #ccc; border-radius: 5px;">
class="delete-group"
        this.tp
style="display: inline-block; width: 400px; padding: 3px; margin-bottom: 10px;">
        this.tp
theme-color btn
id="member-add-%></div></div>"

        this.tp
id="<%=id%>" name="select" style="width: 400px; height: 20px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
        this.tp
style="width: 98%; height: 20px; border: 1px solid #ccc; border-radius: 5px; margin-bottom: 10px;">
name="<%= name%>"
        this.tp
type="checkbox" style="width: 98%; height: 20px;">
    },

    _getDataTypes() {
        this.dataTypes = [
            {
                "key": "String",
                "value": "String"
            },
            {
                "key": "Date",
                "value": "Timestamp"
            },
            {
                "key": "Decimal",
                "value": "Double"
            },
        ],
    },

```

```
        {
            "key": "Number"
            "va
        },
        {
            "ke
            "va
        }
    ]
},
_getTableColumnType
    this.dataType
    {
        "ke
        "va
    },
    {
        "ke
        "va
    },
    {
        "ke
        "va
    },
    {
        "ke
        "va
    },
    {
        "ke
        "va
    },
    {
        "ke
        "va
    }
],
},
_init: function()
    var wrapper =

    var headersLab
    text:
this.constants._labels
    style: 'co
    ));
    wrapper.append

    // Create attr
    $(this.element).append(wrapper);

    this._layout();
```

```

        this._setValue();
    },

    /**
     * Creates Layout and
     *
     */

    _layout: function()
    {
        var headerAttrib
    $(this.tpl_merger_attrib
        var header = thi
        if(this.objectTy
            header = this._o
        }
        headerAttributes

        if(this.objectTy
            var entityRow =
    this._createCustomQueryE
            headerAttributes
        }

        if(this.objectTy
            var entityRow =
            headerAttributes
        }

        var addRowContai
        if(this.objectTy
        || this._uniqueId == "mo
            $(this.element)
        } else {

    $(this.element).append(h
    ontainer);
    }

    },

    /**
     * Creates add more
     *
     */
    createAddButton: fun
        var addContainer
        addContainer.add
        var addButton =
        "buttonLabel" :
    (this.constants._labels[
    constants._labels[this.o
        ]));
        var clearFix = $(this.tpl_div()),

```



```

    _getProcessVariables: function() {
        if (this.objectId) {
            var workflowId = this.workflowId;
            processId).attr("value": workflowId);
        } else {
            workflowId = 0;
        }

        var appId = this.appId;
        appId = (appId ? appId : "default");
        var _proxy = this._proxy;
        var data = {
            applicationId: appId,
            processId: processId
        };

        if (appId) {
            _proxy.doRemoteService(
                + '/findByNameAndScope',
                function(response) {
                    if (response.data) {
                        $.grep(response.data, function(item) {
                            if (item.variable == workflowId) {
                                return item;
                            }
                        });
                    }
                    NBT.util.GenericUtil.showError(
                        false, true, true, true, "Workflow Variables, " + workflowId + " not found.");
                }
            );
        }

        processVariables = [];
        $.each(response.data, function(i, item) {
            processVariables.push(item);
        });
    },

    /**
     * Creates the header row for the table
     *
     * @param {Object} container
     */
    _createHeaderRow: function(container) {
        var headingRow = document.createElement('tr');
        headingRow.className = 'header';
        headingRow.innerHTML = '';

        var keydiv = document.createElement('div');
        keydiv.className = 'key';
        var keyConstant = this.constants._keyConstant;
        this.constants._keyConstant = keydiv;
        var valueConstant = this.constants._valueConstant;
        this.constants._valueConstant = valueConstant;
        var keyLabel = $(this.tpl_header_label({

```

```

        "text":keydiv.cssClass
    }));
    keydiv.append

    if (this.object
        keydiv.css
        var datatyp
$(this.tpl_inline_head
        var datatyp
            "t
            "css
        }));
        datatyped
        datatyped
    } else {
        keydiv.css
    }

    var valuediv =
    var valueLabel
        "text":v
        "cssClass
    }));
    valuediv.append
    if (this.object
        valuediv.c
    } else {
        valuediv.c
    }

    var actiondiv
$(this.tpl_inline_head
    var actionLabel
        "text":t
        "cssClass
    }));
    actiondiv.append
    actiondiv.css

    if (this.object
    if(this._read
        keydiv.
        valuedi
        datatyp

headingRow.append(keydiv)
;
    } else {

headingRow.append(keydiv).append(datatypediv).append(valuediv)
.append(actiondiv);
    }

```

```

    } else if (this._readOnly || this._readOnly) {
        keydiv.css("background-color", "#f0f0f0");
        valuediv.css("background-color", "#f0f0f0");
        headingRow.append(keydiv);
    } else {
        headingRow.append(keydiv);
    }
    return headingRow;
},

_createColumnsHeading: function() {
    var headingRow = $("");
    headingRow.addClass("columns-heading");
    headingRow.css("background-color", "#f0f0f0");

    var keyConstants = this.constants._labelConstants;
    var valueConstants = this.constants._valueConstants;
    var primaryKeyConstants = this.constants._primaryKeyConstants;

    $(this.tpl_inline_heading).appendTo(headingRow);

    var primaryKeydiv = $("");
    primaryKeydiv.text("Primary Key");
    primaryKeydiv.css("background-color", "#f0f0f0");
    headingRow.append(primaryKeydiv);

    var keydiv = $("");
    keydiv.text("Key");
    keydiv.css("background-color", "#f0f0f0");
    headingRow.append(keydiv);

    var valuediv = $("");
    valuediv.text("Value");
    valuediv.css("background-color", "#f0f0f0");
    headingRow.append(valuediv);

    var actiondiv = $("");
    actiondiv.text("Action");
    actiondiv.css("background-color", "#f0f0f0");
    headingRow.append(actiondiv);
}

```

```

    ));
    actiondiv.append(actiondiv.css("display", "block"));
    if (this._ready) {
        keydiv.css("display", "block");
        keydiv.css("display", "block");
    }

    headingRow.append(processVariables);
    v);
    } else {
        headingRow.append(processVariables);
        v).append(actiondiv);
    }

    return headingRow;
},

/**
 * Creates a new table row to the
 * table
 *
 * @param {object} entity
 * @param {string} entityType
 */
_createEntityRow: function(entity, entityType) {
    var rowUniqueId = "row-" + entityType + "-" + entity.name;
    var objName = entity.name;

    var key = (entity.dataType === "string" ? "key" : "value");
    ((entity && entity.name) ? "key" : "value");
    var dataType = entity.dataType;
    entity.dataType : "string";
    var value = entity.value;

    var entityRow = "<tr><td>" + objName + "</td><td>" + value + "</td></tr>";
    entityRow.appendTo("#table-");

    var keydiv = "<div><input type='text' value='" + value + "'></div>";
    if (this.options.processVariables.length > 0) {
        var keyInput = "<input type='text' value='" + value + "'>";
        keyInput.appendTo("#table-");
        if (this.options.processVariables.length > 0) {
            for (var j = 0; j < this.options.processVariables.length; j++) {

```



```

var variableName =
this.options.processVa
var
this.options.processVa
ke
variableName + ' type
'</option>');
}
}
$(keyInput
keyInput.c
this._saveVariableVal
} else {
var rowIdentifi
if (this.objec
rowIdentifi
d+"].key";
d+"].name";
}
var keyInp
box({
"id":
"name"
"value
}));
keyInput.v
keyInput.c
);
NBT.ui.ProcessData.mal
}
keydiv.append
if (this.objec
keydiv.css
var dataTy
ainer());
var dataTy
_textbox({
"id":
"name"
"value
dataType",
}));
$(dataType
//dataType
(this));
dataTypeIn
');
NBT.ui.ProcessData.mal
ut);
dataTypedi
dataTypedi
} else {
keydiv.css
}
var valuediv =
);
var valueInput
x({
"id": "ass
"name": objName+"["+rowUniqueId+"].value",

```

```

        "value": value
    }));
    valueInput.value = value;
    valueInput.focus();
    NBT.ui.Process(valueInput);
    valuediv.append(keydiv);
    if (this.objectId != null) {
        valuediv.append(keydiv);
    } else {
        valuediv.append(keydiv);
    }

    if (this._update) {
        &
    this._readOnly) {
        keyInput.value = value;
        valueInput.focus();
        if (this._update) {
            keydiv.append(keydiv);
            keydiv.append(keydiv);
            valuediv.append(keydiv);
            dataTy
        }
        entityRow.append(keydiv);
        } else {
            keydiv.append(keydiv);
            valuediv.append(keydiv);
            entityRow.append(keydiv);
        }
    } else {
        var deleteButton = document.createElement("button");
        var deleteText = "Delete";
        deleteButton.id = "delete-button";
        deleteButton.innerHTML = deleteText;
        deletediv.append(deleteButton);
        deletediv.append(deleteText);
        deleteButton.onclick = function() {
            this._deleteRow.bind(this);
            if (this._deleteRow) {
                this._deleteRow();
            }
        };
        entityRow.append(keydiv);
        append(deletediv);
        } else {
            entityRow.append(keydiv);
        }
    }

    return entityRow;
},

/**
 * Creates a new row for the entity and append to the
 * table
 *
 * @param {object}

```

[illegible]

```

        entityRow.append(attributeNamediv).append(attributeTypeDiv);
    }

    if(this._readOnly)
        attributeNamediv.append("readOnly");
        attributeTypeDiv.append("readOnly");
        attributeTypeDiv.append(",none");
        attributeTypeDiv.append("none");
    }

    return entityRow;
},

/**
 * Creates a new row in the table
 * @param {object} entity
 */
_createColumnsEntity(entity){
    var rowUniqueID = entity.id;
    var isPrimary = entity.isPrimary ? true : false;
    var name = (entity.name ? entity.name : "");
    //The Key change
    //To avoid breaking
    checking for both type
    var type = (entity.dataType ? entity.dataType : "");
    var entityRow = entityRow.addC
    row");
    var primaryKey = entity.primaryKey;
    var primaryKeyDiv = $(this.tpl_attribute_column)
        .attr("id","attribute_")
        .attr("name","attribute_")
        .attr("type","text");
    primaryKeyDiv.append(primaryKey);
    primaryKeyDiv.css("width","12%");

```

[illegible]

```

        attribu
        attribu
    }

    return entit
},

/**
 * Deletes the s
 *
 * @param {objec
 *     eve
 */
_deleteRow: funct
    if (event &&
        var row = entity-mapper-
row');
        row.remo
        this.sav

    }
},

/**
 * Set values ba
 *
 */
_setValue: funct
    var data = t
    if (this.obj
        if (data
data.headers) {
        if (
            length > 0) {
            s, $("#" +
ner"));
        }
    }
    if(this.
"upload-tableData"){
        if (da
            length > 0) {
            rs, $("#" +
ner"));
        }
    }
    } else if (t
        /*if(dat
data.methodName=="GE
        if ((dat
data.methodName == "
        (data && data.method
        == "mobileflowcanvas"
        if (data.parameters && data.parameters.length
> 0) {

```

```

+ this._uniqueId + "
container"));
    }
    } else if (t
    /*if(dat
data.methodName=="GE
    if ((dat
data.methodName == "
    if (
> 0) {

+ this._uniqueId + "
container"));
    }
    } else if (t
    if (data
data.customAttribute
    this
$("#" + this._unique
container"));
    }
    } else if (t
    this.opt
    this._ge

    if (data
    if (

+ this._uniqueId + "
    }
    } else if(th
    if(data && c
data.outputAttribute

    this.persistCusto
this._uniqueId + "-c
    }
    } else if(th
    //Already Dr
the old code
    if(data && c
    this.p
this._uniqueId + "-c
    }
    //new change
    if(data && c
    this.p
this._uniqueId + "-c
    }
    } else if (this.objectType == "inputParameters") {

```

```

        if (data.inputParameters) {
            data.inputParameters.forEach(function (parameter) {
                if (parameter.type === "text") {
                    this._uniqueId = this._uniqueId + "text" + parameter.name;
                } else if (parameter.type === "checkbox") {
                    this._uniqueId = this._uniqueId + "checkbox" + parameter.name;
                }
            });
        }

        /**
         * Persist the data to the database while
         * persisting the node
         *
         * @param {array} data
         *
         */
        _persistValues(data) {
            $.each(data, function (index, value) {
                var entity = new Entity(value);
                entity.save().bind(this);
            });
        }

        /**
         * Persist the data to the database while
         * persisting the node
         *
         * @param {array} data
         *
         */
        persistCustomQ(query, containerElement) {
            containerElement.find("input").remove();
            $.each(attachments, function (index, value) {
                var attachment = new Attachment(value);
                if (attachment.type === "text") {
                    attachment.save().bind(this);
                } else if (attachment.type === "checkbox") {
                    attachment.save().bind(this);
                }
            });
        }
    },

```



```

        persistColumnValue :
        containerElement) {
            containerElement.remove();
            $.each(attribute, function(i, value) {
                var entity = this._createColumnsEntity(
                    containerElement, value);
            }).bind(this))
        }
    });
});

```

3.3. JAVA Code Details

3.3.1. Installation of Platform JAR Dependencies

Install platform dependencies before we setup any project.

1. Navigate to dependency-repo directory under SupportingFiles in command prompt/terminal.
2. Run the following commands to install the dependencies into maven repository.

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```

mvn install:installFile -Dfile=platform-sp1.jar -
DgroupId=com.qualtrics -DartifactId=platform-sp1 -
Dversion=v5.1.1-sp1 -Dpackaging=jar

```

```
mvn install:install-file -Dfile=node-common-v5.1.1-sp1.jar -
DgroupId=com.quadwave.nbt -DartifactId=node-common -
Dversion=v5.1.1-sp1 -Dpackaging=jar
```

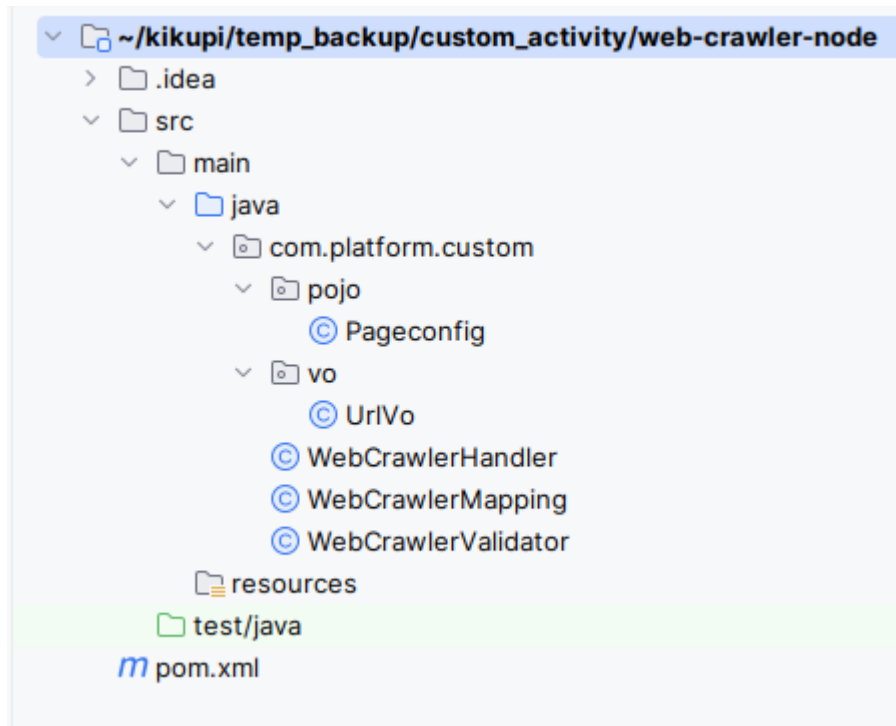


For the latest/updated versions of these JARs, contact platform support team.

3.3.2. Supporting Files for Custom Activity

You need to follow a specific structure for organizing the supporting files that are required for the custom activity configuration.

Custom Activity File Organization:



Refer to provided **JavaJARNode > SupportingFiles** folder for file content and details.

3.3.3. Node JAR Details

The JAR file is mapped in the Node JAR configuration of the Custom Activity.

Node JAR Path

For a custom activity, we will be having a customized java jar, which will have the following classes.

CustomActivityMapping.java

This is the POJO class, we can define all the input and output attributes in the POJO class, which will be mapped to the defined Custom Activity Node. Please find this mapping class in the html-extractor-node project.

CustomActivityValidator.java

This is the validator class, we can define all the mandatory attributes and what type of values and basic validation can be done here in this class.

CustomActivityHandler.java

Whole business logic will be implemented in this class, this class will extend the `RuntimeWorkflowNodeHandler.java` interface, which is from the Platform end. And we will have to override and provide the implementation for the below methods. Please find this handler class in the `html-extractor-node` project.

```
public void init(InstanceContext instanceContext);
```

It will set the `instanceContext`, so that we can have the context of the current instance of the workflow. This context is used to read the variable values at the current moment, and set the variables to the same context, so that it can carry the variable values to the next node in the flow.

```
public HandlerReturn executeNode(NodeContext nodeContext);
```

In this method we will write the business logic, we will read the input data from the Mapping object.

Example:

```
CustomActivityMapping mapping  
= (CustomActivityMapping) nodeContext.getBaseProperty();
```

From mapping object, we can read the values and perform any action in this method. Once we are done with our logic, then we can set back the data to the instance.

Below `variableParser` object can be used to read the variable and parse the variable information.

```
VariableParser variableParser = new  
VariableParser(instanceContext.getWorkflowInstance());
```

`parseVariableName` method will be use to get the variable name from the mapped object.

```
String outputParam =  
variableParser.parseVariableName(mapping.getOutputParam());
```

We can set the value to the instance context as below.

```
instanceContext.setVariableValue(outputParam, "Executed");
```

We can use below method to get the variable value at this moment in the instance level.

```
String variableValue=  
instanceContext.getDeductedValue(mapping.getImagePath());
```

Once we get the variable value, we can do implement our logic here.

Once we are done with the business logic, then we can set back the variable values to the instance context, and same way we have to set the Output, and Error data also to the instance context.

Once we are completed with the setting values to the variables, we can call the below code to continue the flow and we have to return the same object as below. Which means the current node handler is completed and control will reach the next node in the workflow.

```
return new HandlerReturnImpl(null, false);
```

We will have Platform predefined service to get the connection related configuration services, like `DataSource's`, `DMS`, `SMTP`, and other connection configuration services will be available to connect and perform some action. Along with these we can have other services like `Data Entity Services`, `MDM Services`, And Other factory methods will be available.

Based on our requirement we can use the preferred services.

For example, if we want to persist some data through MDMService, we can follow below steps.

```
//Getting the data entity service
DataEntityService dataEntityService = MdmServiceRegistry.getInstance().getDataEntityService();

//Getting the data entity name
DataEntity dataEntity = dataEntityService.getDataEntity("HTMLPage");

// Setting the data entity, so that MDM service can understand the action.
dataEntity.setDataSourceName("HTMLPage");

//Get MDM Service name of the data entity
MdmService mdmService = MdmFactory.getMdmService(dataEntity.getDataSourceName(), dataEntity.getDataSourceName());

//this method will insert the data into the respective table.
mdmService.insert(dataEntity.getName());
JacksonJsonUtil.toJson(dataEntity);
```

3.3.4. Uploading the Node JAR for Custom Activity

This is an additional information for uploading the JAR file in the custom activity.

Node Palette Icon:

Upload SVG Icon in 20X20 size. Sample icon file is attached. This is used for the listing purpose.

Node Icon:

Upload SVG Icon in 40X40 size. Sample icon file is attached. This will be used in the designer.

Node Accordion Icon:

Upload a PNG Icon in 20X20 size. Sample icon file is attached. This will be used as the accordion icon.

4. References

4.1. Supporting Documents

- Configuration Management
- Application Management